Hi

システム・インフラ入門編

H A R M O N I C

i n s i g h t

Harmonic Insight 2026年3月9日

今回の勉強会の目的

本勉強会の学習目標

インフラ・サーバの基礎

インフラとは何か、サーバとは何かが分かる

APIの理解

引数・戻り値・インタフェースの概念を理解する

WEBの仕組み

WEBがどのように動くかの基本構造を学ぶ

システム構成の違い

スタンドアロン型・CS型・WEB型の違いを整理

インフラとは

建物で例えると……

業務アプリ

ソフトウェア (OS等)

ネットワーク

← ITインフラ

ハードウェア

ITインフラとは

システムやサービスを動かすための基盤。
ハードウェア・ソフトウェア・ネットワークなどの総称。

建物で言えば「基礎・柱・電気・水道」にあたる部分。業務システムが動くための土台です。

これがなければアプリもシステムも動かない

サーバーの物理形態 — 3種類

昔は……会社のその辺にも置いてありました

ラックマウント型

ラック（棚）に取り付ける横置き型。データセンターで最も一般的。

大規模

ブレードサーバ型

薄型で高密度に搭載できるタイプ。省スペース・省電力。

超高密度

タワー型

デスクトップPCのような縦置き型。小規模環境で使われる。

小規模

→ 現在は多くの企業がこれらをサーバールームやデータセンターに集約・管理しています

サーバーも、しょせんパソコンとOSと同じ感じ

Windows系

GUI操作
ビジネスユーザに馴染みやすい

ライセンス費用
有償（Microsoft）

Active Directory等の
エンタープライズ機能が豊富

VS

Linux系

コマンドライン操作
オープンソースで無料

自己責任
コミュニティサポートが基本

クラウド・Web系サーバの
デファクトスタンダード

実際の操作画面はパソコンとほぼ同じ。サーバ特有の機能（Webサーバ、DBサーバ等）を提供する点が異なる。

サーバの種類：機能で分類

サーバーとは：クライアント（ユーザ端末）からの要求に応じて機能やサービスを提供するコンピューター

→ 画面（UI/UX）を提供するわけではなく、あくまで「機能」のみを提供する

Webサーバ

HTTPリクエストを処理し、WEBページを配信

DBサーバ

データの保存・検索・管理を担当

メールサーバ

メールの送受信・配信処理を行う

ファイルサーバ

ファイルの共有・保存・管理を提供

提供する機能や役割に応じて様々な呼び方がある — 「何を提供するか」でサーバの名前が決まる

仮想化技術（VM）ーハードとOSの分離

従来

1台に1OS = リソースの無駄

仮想化後

1台で複数VM = リソース効率化

- ハードウェアリソースを効率的に共有
- コスト削減・サーバ統合が可能に
- 開発・テスト環境の素早い構築

コンテナ技術 — さらに軽量な仮想化

VMとコンテナの比較

比較項目	仮想マシン (VM)	コンテナ (Docker)
起動速度	数分	**数秒**
リソース消費	大きい（OS含む）	**小さい（アプリのみ）**
隔離レベル	**高い（OS単位）**	中程度（プロセス単位）
代表例	VMware / Hyper-V	Docker / Kubernetes

コンテナのメリット

- 起動が高速
- リソース消費が少ない
- ポータブル（環境差異なし）
- スケーラブル

物理サーバ → 仮想化（VM） → コンテナ（Docker） → クラウドネイティブ

01

クラウド

ここまでは、会社の中に置いてあるサーバの話……

じゃ、クラウドって何？？

クラウドコンピューティングとは：ネットワーク経由で他者のサーバやアプリケーションを使えるようにした技術

所有不要

自社でサーバを購入・管理する必要がない

従量課金

必要な分だけ利用料を支払って使える

どこからでも

インターネット接続さえあれば利用可能

DX提案では「オンプレミスかクラウドか」が重要な論点になる

クラウドのサービス形態と管理対象

他人の「何」を利用するかでサービスの呼び方が変わる

オンプレミス	IaaS	PaaS	SaaS
アプリ	アプリ	アプリ	アプリ
ミドルウェア	ミドルウェア	ミドルウェア	ミドルウェア
OS	OS	OS	OS
仮想化	仮想化	仮想化	仮想化
ハードウェア	ハードウェア	ハードウェア	ハードウェア
ネットワーク	ネットワーク	ネットワーク	ネットワーク

自社管理　　クラウド事業者が管理

例：IaaS = AWS EC2 / Azure VM　PaaS = Heroku / GAE　SaaS = Gmail / Office365 / Salesforce

02

仕組み・API

ここまでがサーバ、クラウドの話……

03

仕組み・APIの話

最初は数学の関数から……

引数と戻り値（基礎）

数学での関数

$$y = f(x) \qquad Y = f(x, y, z)$$

入力xを受け取り、処理して、結果yを返す

プログラム的に書くと

```
Function(x) {
  z = x * x
  Return z
}
```

引数（ひきすう）

関数に渡す変数のこと。
f(x)のxが引数にあたる。

戻り値（もどりち）

Returnで返される結果の数値。
y = f(x)のyが戻り値にあたる。

引数と戻り値 — IPOの基本構造

入力 → 処理 → 出力（IPO）

Input
（引数）

関数に渡す
変数・データ

→

Process
（処理）

関数の実行
f(x) = x * x

→

Output
（戻り値）

Returnで返す
結果の値

画面操作との対応

ユーザーが画面で値を入力
→ 引数として関数に渡される → 関数が計算処理 → 結果（戻り値）が画面に表示

Function First Design：機能（関数）から考え、UIは後から設計する

引数と戻り値 — 具体的な処理

計算ボタンを押した後、裏で何が起きているか

例1: 2乗計算

入力値: x = 5

→

`f(x) = x * x = 25`

→

戻り値: 25

例2: 合計計算

入力値: x=1, y=2, z=3

→

`f(x,y,z) = x+y+z = 6`

→

戻り値: 6

ユーザーは「ボタンを押す」操作だけ。内部の処理はサーバ側で完結する。
これが「関数」「引数」「戻り値」のシステムへの応用。

クライアントとサーバの定義

クライアント

処理を要求するコンピューター
（ユーザーが操作する側）

サーバ

処理（関数の実行）を行うコンピューター
（計算・処理を担当する側）

このクライアント ↔ サーバの関係がすべてのシステムの基本構造

引数に渡せるのは数字だけではない

文字・画像・音楽・動画など様々なデータを引数として渡せる → 写真を渡して透過処理、動画を渡してCMカット等

WEBだと、こんな感じ

概念	WEBでの実現
関数	サーバ上のプログラム
引数	URLパラメータ / リクエストボディ
戻り値	レスポンス（HTML・JSON等）
クライアント	ブラウザ
サーバ	Webサーバ

URLを入力すると……

① ブラウザにURLを入力

② DNS解決でサーバを特定

③ サーバにリクエスト送信

④ サーバが処理しレスポンス返却

⑤ ブラウザが画面に表示

すべては同じ仕組みの応用 — 関数の概念がWEBの基礎

リクエストの仕組み（GETパラメータ）

GETメソッドによるパラメータ渡し

```
https://example.com/search?q=キーワード
```

↑ URL（サーバのアドレス＋パス）

↑ 引数（パラメータ）

HTTPリクエストメソッドの主な2種類

GET — データ取得

パラメータはURLに付与
URLを共有すれば同じデータを取得可能

POST — データ送信

パラメータはリクエスト本文に含む
フォーム送信やログインで使用

APIとは

API（Application Programming Interface）

アプリケーション同士が通信するための「窓口」。WEBではURLを通じてサーバの機能を呼び出す仕組み。

REST API の主なメソッド

GET	POST	PUT	DELETE
データ取得	データ作成	データ更新	データ削除

コンサルタントの現場で：
「このシステムはAPIを公開していますか？」→ 外部連携の可否を判断する基本質問

APIの例 — Autodesk Construction Cloud

Autodesk Platform Services（APS）

建設クラウドサービスの機能をAPIとして公開 — https://aps.autodesk.com/

- 外部アプリからACCのデータにプログラムでアクセス可能
- 建設現場のプロジェクト・図面データを操作できる
- 基幹システムとの連携や独自ダッシュボード構築に活用

Swagger — APIの仕様書 + テストツール

https://petstore.swagger.io/ でサンプルを体験可能

- 利用可能なAPIの一覧
- 各APIの引数（パラメータ）と使い方
- 戻り値（レスポンス）の具体例

マイクロサービスアーキテクチャ

業務単位で分割された独立したサービスの集合体

モノリシック（一枚岩）

全機能が
1つのプログラム
に結合

マイクロサービス

営業管理 API

施工管理

予実管理 API

原価管理

メリット：各サービスを独立して開発・デプロイ可能 / 障害の影響範囲を局所化 / 技術スタックの柔軟な選択

建設業の基幹システムはモノリシックなものが多く、マイクロサービス化はDXの大きなテーマ

システム構成図とアーキテクチャ図

システム構成図

HW・SW・ネットワーク構成を
物理的・論理的に視覚化した図

アーキテクチャ図

システムの論理的な設計構造を
示す図。企画・設計で重要。

FPTのアプローチ — 5層システム構成

- 5 外部システム連携 — バッチ・リアルタイム同期
- 4 アプリケーション層 — Web版・モバイル版・ダッシュボード
- 3 ビジネスロジック層 — マイクロサービスで業務APIを実現
- 2 データ基盤 — クラウドDB・データレイク
- 1 共通基盤 — アカウント管理・権限管理・SSO

クライアント・サーバ構成（2層型）

昔のスタイル — すべてのPCにアプリをインストール

課題

- 全PCへ再インストールが必要（バージョンアップのたび）
- ライセンス管理・配布コスト大
- サーバに直接接続 → セキュリティ課題

例：CADソフトのインストール型。今でも重いアプリケーションはこの方式が残っている

クライアント・サーバ構成（WEB型・3層型）

現在の主流 — ブラウザですべてを操作

プレゼンテーション層

ブラウザ（クライアント）

↕

アプリケーション層

Webサーバ（処理担当）

↕

データ層

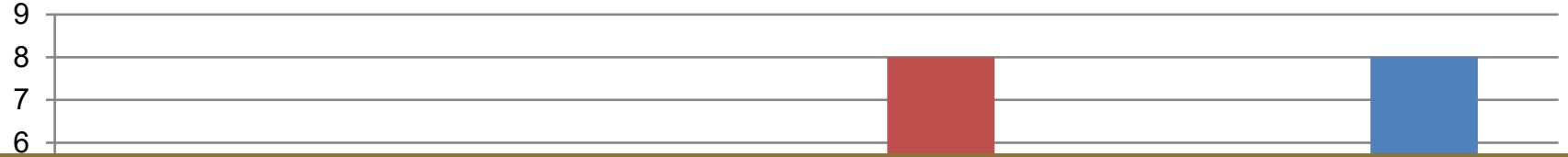
データベースサーバ

メリット

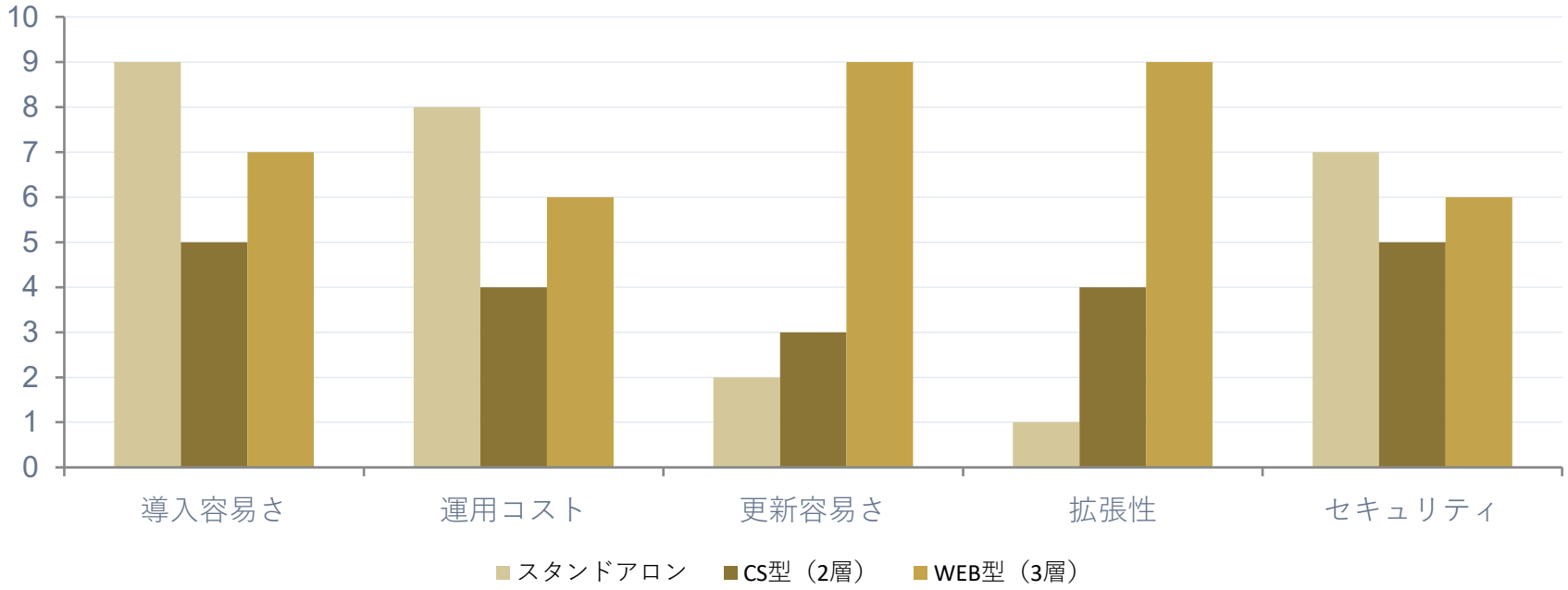
バージョンアップはサーバ側だけ
クライアントへの配布不要
どこからでもアクセス可能

デメリット

ネットワーク経由 → オフライン不可
速度はネットワーク状況に依存
建設現場での通信環境が課題に



まとめ — 3つのアーキテクチャ比較



建設DXでは用途に応じた最適なアーキテクチャ選定が重要

本日の学びの振り返り

インフラ・サーバ

ITインフラ = システムの土台。サーバは機能を提供するコンピュータ。物理→仮想→コンテナと進化。

クラウド

他者のサーバをネット経由で利用。
IaaS/PaaS/SaaSの3分類。所有から利用へ。

API・関数

引数(Input)→処理(Process)→戻り値(Output)。すべてのシステムの基本構造。

アーキテクチャ

2層型→3層型（WEB型）。マイクロサービス。
用途に応じた最適選定が重要。

関数 = API / 引数 = パラメータ / 戻り値 = レスポンス — この対応関係を覚えておこう

Hi

ありがとうございました

H A R M O N I C

i n s i g h t

h-insight.jp